

The Sugar Vault

An Arduino controlled candy & chocolate dispensing machine

MAIN

Start Date: 5/18/2026

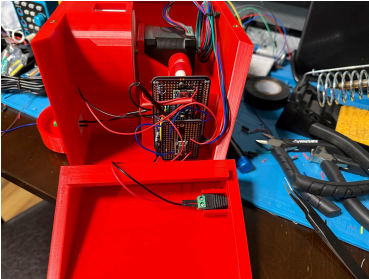
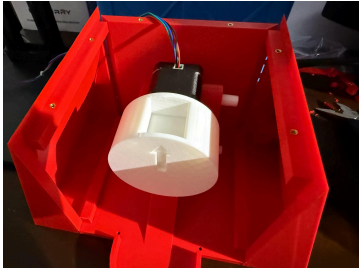
End Date: 6/28/2026

Software Used: Google Docs, SOLIDWORKS, draw.io, Arduino IDE, GitHub, Bambu Studio

Project GitHub: <https://github.com/cchapman9x/the-sugar-vault>

Project Creator:

- Caleb Chapman | Montgomery College | Electrical Engineering

		
Figure 1 Completed Sugar Vault prototype with the acrylic candy-storage tube installed.	Figure 2 Internal electronics and wiring during assembly and testing.	Figure 3 NEMA 17 stepper motor and dispensing wheel installed inside the enclosure.

Project Mission Summary

The Sugar Vault is an Arduino-controlled candy and chocolate dispensing machine designed to dispense candy using a motorized wheel mechanism. The project uses an Arduino Nano, an A4988 stepper motor driver, a NEMA 17 stepper motor, a push button, an LED indicator, a buck converter, and custom 3D-printed mechanical parts.

I created The Sugar Vault for two main reasons. The first reason was to improve my engineering skills by building a complete physical project that combined electronics, programming, CAD, 3D printing, wiring, and mechanical troubleshooting. I wanted a project that was realistic for my current skill level but still challenging enough to teach me new concepts about microcontrollers, motor control, and system integration.

The second reason was more personal. I wanted to build something fun that could eventually be placed in the office where I work so that my coworkers and I could enjoy treats while doing our jobs. The Sugar Vault is not only an engineering project; it is also a creative project based on an older candy dispenser idea I made in high school. I have always wanted to build off of it, so I used this project as the perfect opportunity to do so.

The final version of The Sugar Vault uses a push button as the main input. When the button is pressed, the Arduino controls the A4988 stepper motor driver, which drives the NEMA 17 stepper motor. The motor rotates the dispensing wheel, allowing candy to drop from the storage tube and exit through the dispenser opening. An LED turns on during the dispensing cycle to show that the machine is active.

This project served as an introduction to mechatronics because it required the electrical, mechanical, and software parts of the system to work together.

Project Goals

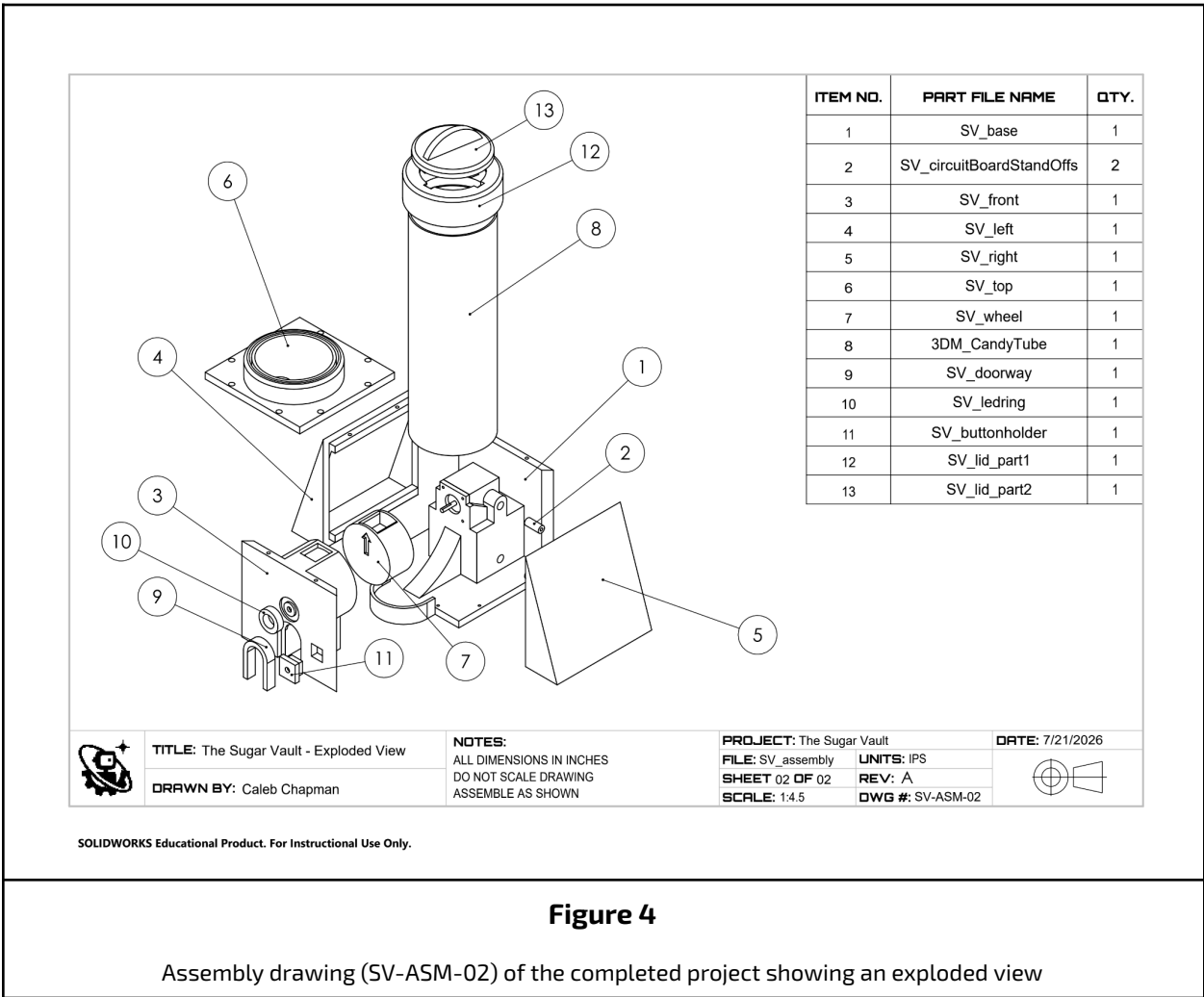
The main goal of The Sugar Vault was to design and build a working Arduino-controlled candy dispenser from start to finish.

The **project goals** were:

- Design a candy dispenser that uses electronic control instead of manual operation
- Use an Arduino Nano as the main controller
- Control a NEMA 17 stepper motor using an A4988 stepper motor driver
- Use a push button as the user input
- Use an LED as a visual indicator during dispensing
- Power the system using an external wall power supply
- Use a buck converter to provide safe logic-level voltage for the Arduino
- Design and 3D print the mechanical parts needed for the dispenser
- Create a wiring diagram for the electronics system
- Test and troubleshoot the dispenser until it worked
- Document the finished project on GitHub

Mechanical Design

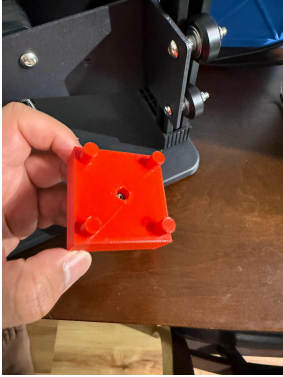
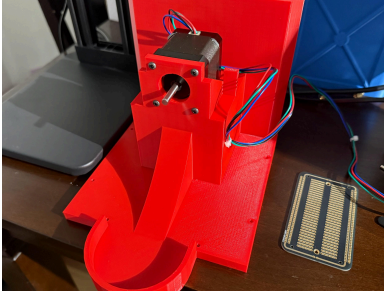
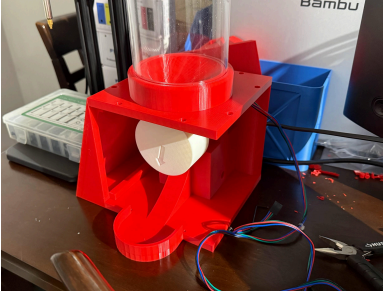
The mechanical system was designed in SOLIDWORKS and manufactured using 3D-printed PETG parts. The main components include the enclosure, acrylic tube holder, dispensing wheel, motor mounting area, front candy opening, button holder, LED holder, and internal electronics supports. Candy is stored inside the clear acrylic tube and falls toward the dispensing wheel. The wheel is connected to the NEMA 17 stepper motor and contains openings that carry candy toward the front exit as it rotates. The surrounding enclosure keeps the wheel aligned and directs the candy through the dispensing opening. The most important mechanical challenge was balancing clearance and support around the dispensing wheel. The wheel needed enough space to rotate without rubbing against the housing, while remaining close enough to prevent candy from falling into the internal compartment. During testing, candy weight sometimes pressed directly against the wheel and created enough friction to stall the motor.



To assemble the machine, I installed heat-set threaded inserts into designated holes so the 3D-printed parts could be fastened together with screws. This was my first time designing parts specifically for heat-set inserts, so I created a separate test piece before working on the actual components. The test helped me practice the installation process and confirm the hole sizes before inserting them into parts with more irregular shapes.

Once the inserts were installed successfully, I began assembling the machine. Super glue was used to secure the acrylic hopper to the upper 3D-printed support and to attach the white lid component to the hopper. One issue I encountered was the white residue that formed after applying super glue to the PETG parts. After researching the problem, I learned that this residue can occur when cyanoacrylate adhesive cures near plastic surfaces. I removed it using WD-40. The residue appeared around the hopper and on the three white front components identified as parts 9, 10, and 11 in Figure 4.

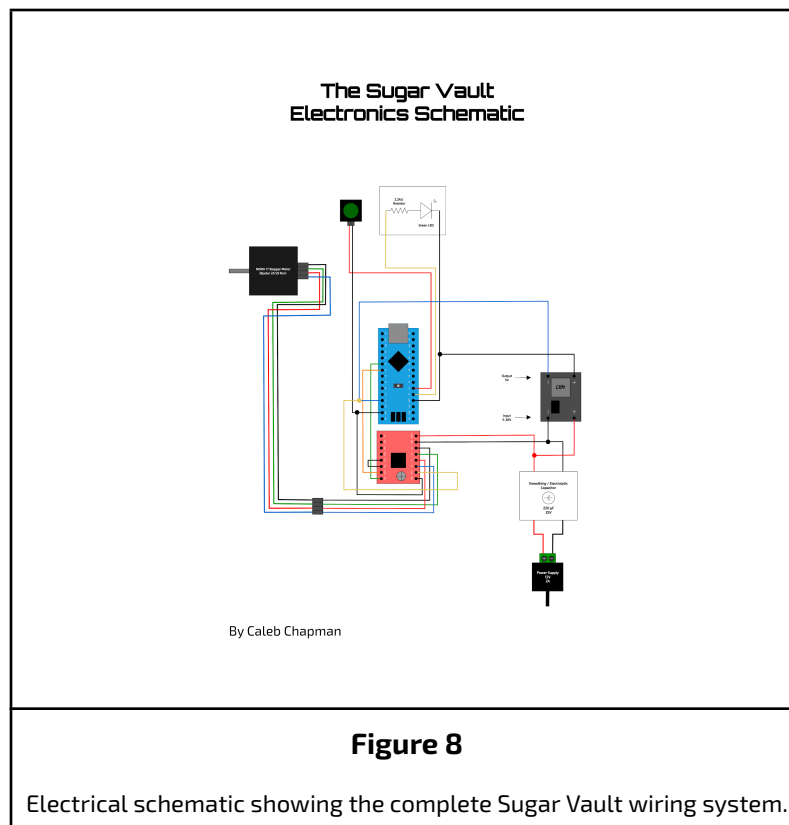
Several parts also required resizing before they fit correctly. This was especially noticeable with the button holder and the circuit-board standoffs. One of the first base structures I printed could not be used because of dimensional errors. These problems showed me the importance of testing tolerances and fit before committing to a full-size print.

		
Figure 5 The first heat-set insert test part that I made	Figure 6 The completed base structure part with the motor attached	Figure 7 The base structure along with the top, wheel, and left wall pieces

Electrical Design

The following schematic shows the electrical connections between the Arduino Nano, A4988 stepper motor driver, NEMA 17 stepper motor, push button, LED, buck converter, capacitor, and 12 V power supply. A bigger version of this schematic can be found on my github linked above under the engineering-design folder.

The electrical system uses a 12 V, 2 A wall power supply as its main power source. The A4988 stepper motor driver uses the external supply to operate the NEMA 17 stepper motor. A DC-DC buck converter reduces the input voltage to approximately 5 V for the Arduino Nano and other low-voltage components. The Arduino Nano acts as the controller. It receives the push-button signal, controls the LED indicator, and sends STEP and DIR signals to the A4988 motor driver. The STEP signal causes the motor to move, while the DIR signal determines the direction of rotation. A 220 μ F electrolytic capacitor is connected near the motor-driver power input to help reduce voltage fluctuations. The green LED uses a 2.2 k Ω current-limiting resistor. All components share a common ground so the Arduino, motor driver, power supply, and buck converter use the same electrical reference.



Before soldering the final circuit, I prototyped the system extensively on a breadboard. I first used the breadboard setup to understand how the stepper motor and A4988 driver operated together. Once the motor was running, I adjusted its direction and speed so it would work correctly with the dispensing mechanism. To make the wheel's movement easier to observe, I added an arrow indicator to the dispensing wheel. During the first tests, the motor movement was rough and inconsistent. I improved its operation by adjusting the timing between step pulses in the Arduino code until the wheel rotated more smoothly and at an appropriate speed for dispensing candy.

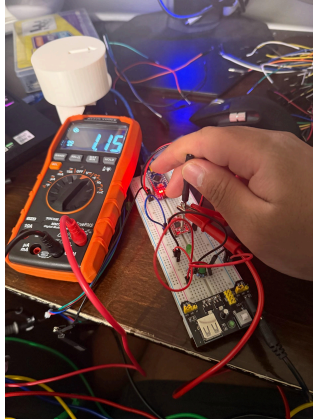


Figure 9

Breadboard prototype of the circuit



Figure 10

The motor & wheel turning test

To create the final prototype board, I used ElectroCookie solderable breadboards and assembled the circuit using a soldering iron. Because I had limited soldering experience, I encountered several problems during the process. For example, I damaged one of the buck converters during my first attempt. Fortunately, I had ordered multiple units, which allowed me to replace it and try again. I also made several incorrect solder connections while transferring the circuit from the breadboard to the prototype board. These mistakes led me to create a more detailed wiring diagram that I could follow during assembly. The ElectroCookie boards were especially useful because their internal connection layout is similar to a standard breadboard, making it easier to transfer the tested circuit. The circuit-board standoffs used in the final enclosure were designed specifically to fit the ElectroCookie prototype boards.

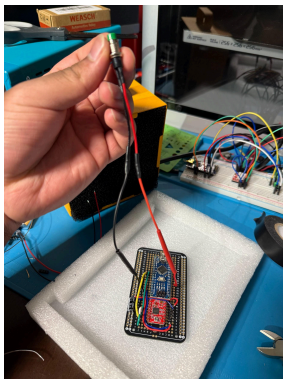


Figure 11

Soldered ElectroCookie prototype board during wiring.

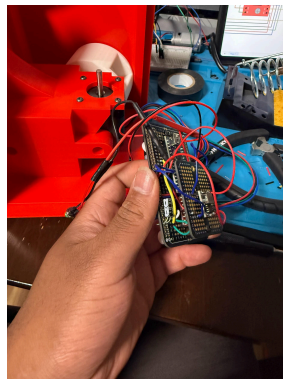


Figure 12

Completed prototype board before installation.

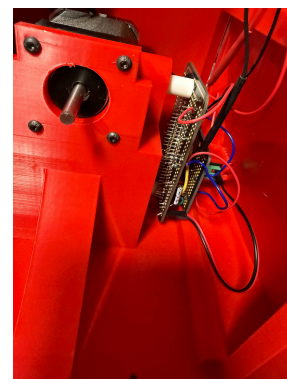


Figure 13

Prototype board mounted inside the enclosure.

Code & Control Logic

```
const int dirPin = A2;
const int stePin = A3;
const int buttonPin = 3;
const int ledPin = 2;

void setup() {
  pinMode(dirPin, OUTPUT);
  pinMode(stePin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);

  digitalWrite(dirPin, LOW);
  digitalWrite(ledPin, HIGH);
  digitalWrite(ledPin, LOW);
}

void loop() {
  bool dispenseCandy = digitalRead(buttonPin) == LOW;
  if (dispenseCandy) {
    digitalWrite(ledPin, HIGH);
    for (int i=0; i < 200; i++) { // this is 200 steps
      digitalWrite(stePin, HIGH);
      delayMicroseconds(7000);
      digitalWrite(stePin, LOW);
      delayMicroseconds(7000);
    }

    digitalWrite(ledPin, LOW);
  }
}
```

Code Breakdown

Pin Initialization

```
const int dirPin = A2;
const int stePin = A3;
const int buttonPin = 3;
const int ledPin = 2;
```

At the top of the code, I initialized 4 variables, each representing a pin. Please note that this code was originally made using a breadboard setup, so some wire colors may be different from what is shown on the final schematic.

- **dirPin** - The dirPin is set to pin A2 on the arduino. Pin A2 is connected to the DIR pin of the A4988 stepper motor driver. It should be noted that pin A2 is an analog pin but can also be used as a digital pin. The arduino sends either a HIGH or LOW digital output signal to this pin which the A4988 then receives. The signal tells the A4988 driver which direction to rotate the stepper

motor only when STEP pulses are sent. The A4988 changes the direction of the motor by energizing the motor coils in different orders. This allows the driver to control both CW and CCW movement of the NEMA 17 motor.

- **stePin** - The stePin is set to pin A3 on the arduino. Pin A3 is connected to the STEP pin of the A4988 stepper motor driver. The arduino also sends HIGH and LOW digital output signals to this pin which the A4988 receives. Though the DIR pin controls the direction of rotation of the stepper motor, the STEP pin is what controls / triggers the movement altogether. Without the STEP signal, the DIR signal can only set the direction but the motor won't physically move. This can be further explained with a **timing trace**! So each pulse of the STEP pin tells the A4988 to move the motor one step in the direction given by the DIR pin signal.
- **buttonPin** - The buttonPin is set to pin 3, which is pin D3 on the arduino. Pin D3 is connected to the red / signal wire of the green momentary push button which acts as the user input in this system. The buttonPin is set to INPUT_PULLUP in the code which means the arduino will use its pull-up resistor. When we have a button connected to an Arduino, it needs a clear voltage state or digital state, either HIGH or LOW, in order to be read correctly. If the button input is not connected to a clear state, it can "float," which means the Arduino may read random HIGH or LOW values because the pin is picking up electrical noise instead of a clear digital signal. INPUT_PULLUP fixes this by turning on a small internal resistor inside the Arduino that weakly connects the input pin to 5V. This means that when the button is not pressed, the input pin reads HIGH. In this project, D3 is internally connected to 5V through the pull-up resistor because of the INPUT_PULLUP setting in the code. When the button is pressed, the button pulls D3 down to GND, so the Arduino reads LOW. This is why the code checks if the button reads LOW in order to trigger the motor. When the button is pressed, it pulls pin D3 LOW, creating the LOW digital input state that the arduino reads. This is how the arduino knows to send the digital output signals to pin A3 which are received by the A4988 driver which then triggers the motor. Note also that the D in D3 means that this pin is for digital signals which is why after learning more about the formatting of the arduino, I connected it here.

	○ Not Pressed = HIGH ○ Pressed = LOW ● ledPin - The ledPin is set to pin 2, which is pin D2 on the Arduino. Physically, this pin connects to a 2.2kΩ resistor, which then connects to the actual LED. The resistor limits the amount of current flowing through the LED so the LED and Arduino pin are protected. When the button is pressed, the LED turns on, indicating that the dispensing wheel is turning and dispensing candy. It turns off once the wheel has completed its programmed rotation. The LED is an output device because the Arduino sends a digital signal to control whether it is on or off.
The setup() Function	In Arduino programming, the setup() function is used as the one-time startup section of the program. It runs once when the Arduino first turns on, resets, or starts the sketch, which is the Arduino program. It is basically the function where you can set default values, set pin modes, and initialize libraries. This is the <u>preparation</u> function for the program. <pre>pinMode(pin, MODE);</pre> ● pinMode() - The pinMode() function is used for setting the mode of a pin. It allows us to control the behavior of each pin and determine if we are inputting data with the pin or outputting it. ○ dirPin & stePin - These pins are set to OUTPUT mode because the arduino uses them to output digital signals to the A4988 motor driver. ○ buttonPin - The buttonPin is an input pin because it receives input from the user to activate the motor. However, the button also needs a clear default state so the pin does not float. Because of this, the pin mode is set to INPUT_PULLUP instead of just INPUT. This allows the Arduino to use its internal pull-up resistor, making the button read HIGH when it is not pressed and LOW when it is pressed. ○ ledPin - This pin is set to OUTPUT because it is used to convey that the dispensing wheel is on to the user. ● digitalWrite() - This function is used to control the digital output signal of the output pins. It will tell the pins to go either HIGH or LOW. When an output pin is set to HIGH, the Arduino sends voltage out of that pin.

	When an output pin is set to LOW, the Arduino brings that pin down to 0V/GND. In this project, digitalWrite() is used to set the direction signal for the motor driver and to turn the LED on or off. ○ dirPin - We set the dirPin to LOW because the digital output signal controls the direction of rotation. A LOW signal means that the motor will turn CCW while a HIGH signal means the motor will turn CW. I wanted the motor to turn CCW. ○ ledPin - HIGH turns the LED on and LOW turns it off. So we use this function simply to control the LED.
The loop() Function	In arduino programming, the loop() function is used as the main repeating part of the program. After the setup() function runs once, the loop() function that runs on repeat as long as the arduino is powered. The loop in the program does the following: 1. Read the value of the button and check for a LOW signal 2. If the button is pressed, meaning the digital signal is LOW, do the following: a. Turn the LED on b. Make the motor do one full 360° turn c. Once the motor is finished turning, turn off the LED 3. Repeat Lets look inside the loop() function ● dispenseCandy - This is a Boolean variable that stores either true or false. It gets its value from the expression digitalRead(buttonPin) == LOW. The digitalRead() function first reads the state of buttonPin. Since the button uses INPUT_PULLUP, pressing the button makes the pin read LOW. If digitalRead(buttonPin) returns LOW, then the comparison becomes true, meaning dispenseCandy is set to true and the dispensing cycle can begin because that is then plugged into the if statement. ● Inside the if statement, we first turn on the LED to indicate that the dispensing wheel is moving. ● We then turn the wheel 360° using a for loop. A single motor step is about 1.8°. I needed the motor to rotate 360°, so this involved a little bit of math. I divided 360 by 1.8, which gave me 200. This meant that the motor needed 200 steps in order to complete one full 360° rotation. Because I knew the exact number of iterations needed, I decided that using a for loop would be best for

	making the motor complete 200 steps. • I set $i = 0$, and as long as i was less than 200, the motor would keep stepping while the for loop kept track of the number of steps. In order to make the motor step, I used <code>digitalWrite()</code> to set the STEP pin HIGH, starting the STEP pulse. I then used <code>delayMicroseconds(7000)</code> to give the motor time between pulses, which makes the movement smoother and steadier. I wanted the motor to move slowly, so I set the delay to 7000 microseconds. This is equal to 0.007 seconds, or 7 milliseconds. I then set the STEP pin LOW to end that single step pulse and added another 7000 microsecond delay before the next pulse. The for loop repeated this process 200 times, resulting in a smooth 360° turn. • Once the for loop ends, the LED is turned off, indicating that the wheel has stopped moving.
--	---

Final Report & Project Reflection

The final version of The Sugar Vault successfully dispenses candy using a push button-controlled stepper motor system. The project combines an Arduino Nano, A4988 stepper motor driver, NEMA 17 stepper motor, buck converter, external power supply, LED indicator, a clear acrylic tube as the hopper, and custom 3D-printed parts into one complete working prototype.

This project helped me practice several important engineering skills. I learned more about wiring electronic components together, using an external power supply, controlling a stepper motor, working with a motor driver, designing parts in CAD, preparing files for 3D printing, how to split power for different components, and troubleshooting a full physical system.

One of the most important lessons from this project was that a working engineering design is not only about the electronics or the code. The mechanical design matters just as much. Even when the circuit and code are correct, the machine still has to physically move without jamming, rubbing, or stalling. During testing, The Sugar Vault worked, but I noticed that the dispensing wheel could occasionally get stuck when too much candy weight pressed down on it. This appears to be a mechanical load and friction issue rather than a software issue. The motor and code can rotate the wheel, but the weight of the candy can sometimes create enough resistance to stall the mechanism.

Overall, The Sugar Vault was completed as a functional Arduino-based mechatronics project. It achieved the main goal of creating a motorized candy dispenser while also showing clear areas for future mechanical improvement.

Lessons & Skills Learned

The Sugar Vault showed me that a working electromechanical system depends on the mechanical, electrical, and software designs working together. Even when the wiring and code function correctly, mechanical friction, alignment, material tolerances, and applied loads can still prevent the machine from operating reliably. The project also taught me the importance of testing individual subsystems before combining them. Testing the motor, button, LED, power system, and dispensing mechanism separately made it easier to identify whether a problem came from the code, electronics, or mechanical design.

Skills practiced during the project included:

- SOLIDWORKS part and assembly modeling
- 2D Engineering drawings
- 3D printing and PETG fabrication
- Arduino programming
- Stepper motor control
- A4988 motor-driver wiring
- Voltage regulation using a buck converter
- Electrical schematic creation
- Mechanical and electrical troubleshooting
- GitHub project documentation
- How to design heat-set inserts

Known Limitations & Future Improvements

The main limitation of the current design is the weight of the candy pressing down on the dispensing wheel. During testing, I noticed that the motor would sometimes stall after the button was pressed, preventing the wheel from turning. After investigating the issue, I concluded that too much candy weight was resting directly on the dispensing wheel, creating enough friction and resistance to prevent the motor from rotating it consistently. The shape of the candy may have also contributed to the wheel getting stuck, especially when combined with the weight of the candy pressing down on the mechanism.

In the future, I plan to address this issue by designing a slanted holding piece or **load-relief baffle** inside the hopper. This part would help prevent most of the candy weight from sitting directly on top of the dispensing wheel, allowing the wheel to rotate more reliably during dispensing.

Even with this limitation, the project still works as intended. However, the design could benefit from further revisions and improvements.

Future improvements I would like to implement include:

- Adding a load-relief baffle above the dispensing wheel
- Reducing friction between the dispensing wheel and the housing
- Improving the geometry of the wheel pockets
- Testing different candy sizes and shapes
- Improving reliability when the candy tube is filled with more weight
- Creating a cleaner internal wiring layout
- Designing a custom PCB for the electronics system
- Improving the exterior finish of the enclosure
- Using a different filament for the final version (PETG is a very stringy filament)

Reference Materials & Resources:

- *Arduino INPUT_PULLUP Explained (pinMode)*
https://roboticsbackend.com/arduino-input_pullup-pinmode/
- *Control LED with Push Button – Arduino Tutorial*
<https://www.circuits-diy.com/control-led-with-push-button-arduino-tutorial/>
- *Arduino Nano Pinout – Full Guide for Beginners*
<https://leecuriosity.com/arduino-nano-pinout-full-guide-for-beginners/>
- *Control a NEMA 17 Stepper Motor with A4988 Driver and Arduino - Full Guide*
<https://www.youtube.com/watch?v=wcLeXXATCR4&t=810s>
- *Simple 3D Design for Heat Set Inserts in Tinkercad*
https://www.youtube.com/watch?v=6YO_21WFFzI